Catalog, Checkout & Orders

Module specification: backend, frontend, schemas, REST contracts and sequence diagrams

1. Catalog & Pricing

HLD

Catalog owns sellable product definitions and immutable price versions. Checkout owns time-bound purchase sessions and the immutable order snapshot. A price edit creates a new version or new price record; historical orders never recalculate from today's catalog.

LLD and rules

- Product lifecycle: draft -> active -> archived. Only active products and active prices can be purchased.
- Price models: one_time, subscription, usage and credit_pack. Model-specific fields are required and mutually validated.
- Checkout validates tenant ownership, product status, price status, currency, quantity limits, discount eligibility and configured tax calculation.
- Server computes all amounts. Ignore client-supplied amount/tax/discount values.
- Session lifecycle: open -> completed | expired | cancelled. Completed sessions are immutable.
- Use an idempotency key for session creation. Same key + same normalized payload returns the original result; different payload returns 409.
- Snapshot product name, price version, unit amount, quantity, discount/tax components and currency into the order.

MongoDB schema and indexes

```
type ProductDoc = {
  _id: ObjectId; tenantId: ObjectId; publicId: string;
  name: string; slug: string; description?: string;
  productType: "digital_service"|"download"|"license";
  status: "draft"|"active"|"archived";
  fulfillment: { type: "entitlement"|"download"|"license"|"custom"; config: Record<string, unknown> };
  metadata: Record<string, unknown>;
  createdAt: Date; updatedAt: Date;
};
// Unique: { tenantId: 1, publicId: 1 }
// Unique: { tenantId: 1, slug: 1 }

type PriceDoc = {
  _id: ObjectId; tenantId: ObjectId; productId: ObjectId; publicId: string;
  version: number; model: "one_time"|"subscription"|"usage"|"credit_pack";
  currency: string; unitAmountMinor?: number;
  interval?: "day"|"week"|"month"|"year";
  usage?: { metric: string; unit: string; aggregation: "sum"|"max"|"last"; pricingModel: "flat"|"per_unit"|"tiered" };
  creditGrant?: { quantity: number; expiryDays?: number };
  status: "active"|"archived"; createdAt: Date;
};
// Unique: { tenantId: 1, publicId: 1 }
// Index: { tenantId: 1, productId: 1, status: 1 }
```

DTOs

```
export type CreateProductDTO = {
  name: string; slug: string; description?: string;
  productType: "digital_service"|"download"|"license";
  fulfillment: { type: "entitlement"|"download"|"license"|"custom"; config?: Record<string, unknown> };
  metadata?: Record<string, unknown>;
};
export type ProductDTO = CreateProductDTO & {
  id: string; status: "draft"|"active"|"archived";
  createdAt: string; updatedAt: string;
};
export type CreatePriceDTO = {
  productId: string; model: "one_time"|"subscription"|"usage"|"credit_pack";
  currency: string; unitAmountMinor?: number;
  interval?: "day"|"week"|"month"|"year";
  usage?: { metric: string; unit: string; aggregation: "sum"|"max"|"last"; pricingModel: "flat"|"per_unit"|"tiered" };
  creditGrant?: { quantity: number; expiryDays?: number };
};
```

```
export type CreateCheckoutSessionDTO = {
  customerId: string; items: Array<{ priceId: string; quantity: number }>;
  successUrl: string; cancelUrl: string; affiliateCode?: string;
};
export type CheckoutSessionDTO = {
  id: string; status: "pending"|"completed"|"expired"|"cancelled";
  currency: string; totalMinor: number; checkoutUrl: string; expiresAt: string;
};
```

REST contracts

Method / route	Request	Success	Errors
POST /api/v1/products	CreateProductDTO	201 ProductDTO	400, 401, 403, 409
GET /api/v1/products	limit, after, status?	200 Page	400, 401, 403
GET /api/v1/products/:id	Path public ID	200 ProductDTO	401, 403, 404
PATCH /api/v1/products/:id	Partial allowed fields + expectedVersion	200 ProductDTO	400, 403, 404, 409
POST /api/v1/prices	CreatePriceDTO	201 PriceDTO	400, 403, 404, 409
POST /api/v1/checkout/sessions	CreateCheckoutSessionDTO + Idempotency-Key	201 CheckoutSessionDTO	400, 401, 403, 409, 422, 429
GET /api/v1/checkout/sessions/:id	Path public ID	200 CheckoutSessionDTO	401, 403, 404

Validation

- Product name 1–160 characters; slug normalized and unique within tenant; description bounded.
- Quantity must be a positive integer within configured maximum. Reject floating-point quantities unless a product model explicitly supports them.
- Currency must be supported by platform configuration. Amount must be a safe integer in the configured minor unit.
- successUrl/cancelUrl must match allowed schemes and tenant URL policy; prevent open redirects.
- Only compatible price fields are accepted for each model; reject unknown fields.

UI flow and components

Screen	Components	State strategy / edge cases
Product list	ProductTable, StatusFilter, Search, Pagination	Server state via TanStack Query or existing CPG standard; URL owns filters and cursor.
Product editor	ProductForm, FulfillmentSelector, MetadataEditor	Local form state; validate on blur and submit; preserve draft on API failure.
Price editor	PriceModelSelector, CurrencyInput, IntervalFields, UsagePricingFields	Conditional schema per model; prevent incompatible fields; show immutable price/version warning.
Publish confirmation	PublishDialog, PriceSummary	Fetch current server state before publish; handle concurrent edit with 409.
Checkout preview	CheckoutPreview, DiscountSummary	Server-calculated quote is authoritative; never calculate final amount only in browser.
Checkout result	PendingState, SuccessState, FailureState	Poll or subscribe to server status; redirect alone does not prove payment.

Full-stack sequence

```
sequenceDiagram
  participant UI as Product Editor
  participant SDK as kubera.catalog
  participant API as Express
  participant S as CatalogService
  participant DB as MongoDB
  UI->>SDK: products.create(dto)
  SDK->>API: POST /products
  API->>API: Authenticate + tenant scope + validate
```

```
  API->>S: createProduct(tenant, dto)
  S->>DB: Insert product
  DB-->>S: Product document
  S-->>API: Domain product
  API-->>SDK: 201 ProductDTO
  SDK-->>UI: Typed result
```

2. Orders

LLD

- Order is the commercial purchase snapshot, not the payment attempt. One order may have multiple payment attempts and partial/full refunds.
- Order state and fulfillment state are separate. A paid order can have fulfillmentStatus=failed_pending_retry.
- Snapshot price, item description, tax, discount, currency and totals at order creation.
- Order transitions are validated in a domain service; administrative overrides require reason and audit entry.
- Never delete paid orders. Cancel/expire only according to policy and record all related attempts.

Schema and DTO

```
type OrderDoc = {
  _id: ObjectId; tenantId: ObjectId; publicId: string; customerId: ObjectId;
  items: Array<{ productId: ObjectId; priceId: ObjectId; productNameSnapshot: string;
    quantity: number; unitAmountMinor: number; lineTotalMinor: number }>;
  subtotalMinor: number; discountMinor: number; taxMinor: number; totalMinor: number;
  currency: string; status: "pending"|"awaiting_payment"|"paid"|"partially_refunded"|"refunded"|"cancelled"|"expired";
  fulfillmentStatus: "not_required"|"pending"|"processing"|"completed"|"failed";
  idempotencyKey?: string; createdAt: Date; updatedAt: Date;
};
// Indexes: unique {tenantId, publicId}; {tenantId, customerId, createdAt desc}
// Partial unique idempotency index scoped to the operation and tenant.
export type OrderDTO = {
  id: string; customerId: string; items: Array<{productId:string; priceId:string;
    name:string; quantity:number; unitAmountMinor:number; lineTotalMinor:number}>;
  subtotalMinor:number; discountMinor:number; taxMinor:number; totalMinor:number;
  currency:string; status:string; fulfillmentStatus:string; createdAt:string;
};
```

Endpoints and UI

Route	Contract
GET /api/v1/orders	Tenant-scoped list; filters customerId, status, createdFrom/To; cursor pagination.
GET /api/v1/orders/:id	Returns OrderDTO with payment summaries, never provider secrets.
POST /api/v1/orders/:id/cancel	Allowed only before paid/captured according to provider and policy; returns 202 if async.
POST /api/v1/orders/:id/retry-fulfillment	Operator-only, audited, idempotent operation.

UI: Orders table -> order detail -> line items/totals -> payment attempts -> refunds -> fulfillment timeline -> audit history. Filters and pagination live in URL/server state; selected row and open tabs are local state. Show stale/pending badges and refresh on webhook-driven invalidation.

Sequence

```
sequenceDiagram
  participant UI as Order Detail
  participant SDK as kubera.orders
  participant API as Express
  participant S as OrderService
  participant DB as MongoDB
  UI->>SDK: orders.get(orderId)
  SDK->>API: GET /orders/:id
  API->>S: getForTenant(tenantId, orderId)
  S->>DB: Find by tenantId + publicId
  DB-->>S: Order snapshot
  S-->>API: OrderDTO
  API-->>SDK: 200
  SDK-->>UI: Render timeline and totals
```